Risk Alert Classifier

High-Risk Customer Prediction

1. Introduction

In this project I built an early-warning system for a digital bank. The bank wants to find high-risk customers, who are likely to default on payments or commit fraud, before the loss happens. The dataset is highly imbalanced because only about 12% of the customers are risky, so a model that is judged only by accuracy would give a false feeling of success.

To solve this I built a complete classification pipeline: KNN imputation for the missing values, a Logistic Regression baseline, four imbalance-handling techniques (Under-Sampling, Over-Sampling, SMOTE and ADASYN), Decision Tree and Random Forest models, and hyperparameter tuning with Randomized Search and Grid Search. I compared all the models on the same stratified train–test split and the same metrics, and I selected the final model using the business rule that a missed risky customer (false negative) costs more than a false alarm (false positive). At the end I saved the best model and made a Streamlit app where anyone can enter customer details and get a risk prediction.

2. Part A: Theory Concepts

2.1 What is Logistic Regression and why is it suitable for classification? (Q1)

Logistic Regression is a supervised model that first calculates a linear score $z = b0 + b1 \cdot x1 + \ldots + bn \cdot xn$ and then passes it through the sigmoid function $\sigma(z) = 1 / (1 + e^{(-z)})$. The output is a probability between 0 and 1. If the probability is above a threshold (0.5 by default), the customer is classified as class 1, which means high risk. It is suitable for classification because it gives a probability, it is fast, it works well as a baseline and its coefficients are easy to explain: each one shows how much a feature pushes the odds of risk up or down.

2.2 Classification performance metrics and why accuracy alone is not enough (Q2)

All classification metrics are calculated from the confusion matrix, which counts the true negatives (TN), false positives (FP), false negatives (FN) and true positives (TP). Here the positive class is High Risk. Accuracy = (TP + TN) / total, but on imbalanced data it is misleading. For example, if only 12% of the customers are risky, a model that says “low risk” for everybody gets about 88% accuracy and still catches zero risky customers. That is why I also use Precision, Recall, F1-Score and AUC-ROC.

Actual \ Predicted	Low Risk (0)	High Risk (1)
Low Risk (0)	TN – correct	FP – Type-I error
High Risk (1)	FN – Type-II error	TP – correct

2.3 Type-I and Type-II errors in risk prediction (Q3)

A **Type-I error** (false positive) happens when a safe, low-risk customer is flagged as high risk. The bank wastes review effort and can lose goodwill. A **Type-II error** (false negative) happens when a really risky customer is predicted as low risk. The default or fraud goes unnoticed and the bank loses money. For an early-warning system the Type-II error is normally the more expensive one.

2.4 Precision, Recall, F1-Score, TPR and FPR (Q4)

- **Precision** = TP / (TP + FP): of all customers flagged as high risk, how many really are.
- **Recall (TPR, sensitivity)** = TP / (TP + FN): of all truly high-risk customers, how many I caught.

- **F1-Score** = 2 · Precision · Recall / (Precision + Recall): the harmonic mean, which balances the two.
- **FPR** = FP / (FP + TN): the share of safe customers that were wrongly flagged.

Example: out of 1,000 customers, 100 are risky. If a model catches 80 of them (TP = 80, FN = 20) and raises 10 false alarms (FP = 10, TN = 890), then Accuracy = 97%, Precision = 88.9%, Recall = 80% and FPR = 1.1%. Accuracy looks great, but 20 risky customers are still missed, so Recall is the metric I watch most.

2.5 What is AUC-ROC and how does it help? (Q5)

The ROC curve plots the True Positive Rate (y-axis) against the False Positive Rate (x-axis) for every possible threshold. AUC is the area under this curve: 0.5 means random guessing and 1.0 means perfect separation. It shows how well a model ranks risky customers above safe ones, without depending on one fixed threshold, so it is a good way to compare different models.

2.6 Why does imbalanced data create problems? (Q6)

When the model sees very few high-risk examples, it can reduce its overall error simply by favouring the majority class. The result is high accuracy but low recall for the minority class, a decision boundary pushed away from the minority, and metrics that look better than the real performance. I handle this with resampling (Under-Sampling, Over-Sampling, SMOTE, ADASYN), class weights, threshold tuning and by using better metrics.

3. Part B: Dataset and Preparation

3.1 Input features and target variable (Q7)

The dataset has 4,600 customers and 19 columns with demographic, credit-behaviour and transaction-activity information. The target variable is risk_status (0 = Low Risk, 1 = High Risk). Only 557 customers (12.1%) are high risk, against 4,043 low-risk customers, so the data is clearly imbalanced. I dropped customer_id because it is only an identifier, and I converted last_transaction_date into a numeric feature, days_since_last_txn. This gave 17 input features: 14 numeric and 3 categorical (gender, region and employment_type).

Feature group	Columns
Customer demographics	age, gender, region, employment_type, annual_income_inr
Credit behaviour	credit_score, credit_utilization_ratio, missed_payments_12m, avg_late_payment_days, cash_advance_count_6m, debt_balance_inr
Transaction / account activity	monthly_transaction_count, monthly_spend_inr, days_since_last_txn, complaints_last_6m, failed_login_attempts_3m, account_tenure_months

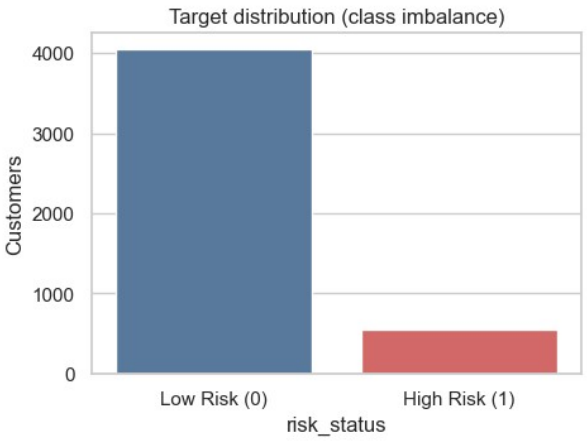


Figure 1: Target distribution (class imbalance)

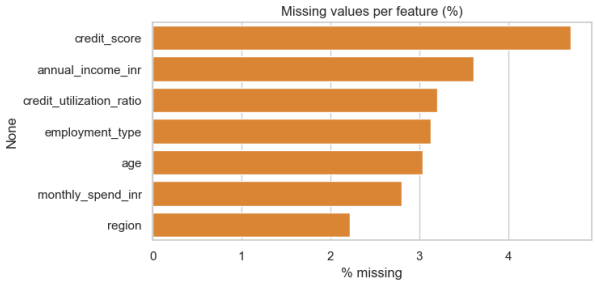


Figure 2: Missing values per feature (%)

3.2 Train–test split with the same class distribution (Q8)

I split the data 80/20 with stratify = y and random_state = 42, which gave 3,680 rows for training and 920 rows for testing. Stratification keeps the high-risk share almost the same in all parts, so the small minority class cannot end up under-represented in the test set.

Data set	Rows	Low Risk	High Risk	High-risk share
Full data	4,600	4,043	557	12.11%
Train	3,680	3,234	446	12.12%
Test	920	809	111	12.07%

3.3 Missing values and KNN Imputer (Q9)

Seven columns had missing values, between 2% and 5% each (850 missing cells in the training set).

Column	Missing count	Missing %
credit_score	216	4.70
annual_income_inr	166	3.61
credit_utilization_ratio	147	3.20
employment_type	144	3.13
age	140	3.04
monthly_spend_inr	129	2.80
region	102	2.22

For the numeric columns I used StandardScaler followed by KNNImputer with 5 neighbours. KNN uses distances, so the features must be on one scale first. Each missing value is filled from the 5 most similar customers, which is better than a simple mean because it uses the other columns too. For the categorical columns I replaced the missing values with a separate “Unknown” category and then applied One-Hot Encoding. Everything was fitted on the training set only, so no information leaks from the test set. After this step there were 0 missing values and 29 model-ready columns. For example, missing credit scores were filled with realistic values such as 724, 717 and 679.

4. Part C: Baseline Classification Model

4.1 Logistic Regression baseline (Q10)

I trained Logistic Regression (max_iter = 1000) on the processed training data as the baseline. The coefficients agree with real banking logic. Risk goes up with avg_late_payment_days, credit_utilization_ratio, debt_balance_inr, missed_payments_12m and failed_login_attempts_3m, and it goes down with annual_income_inr and credit_score.

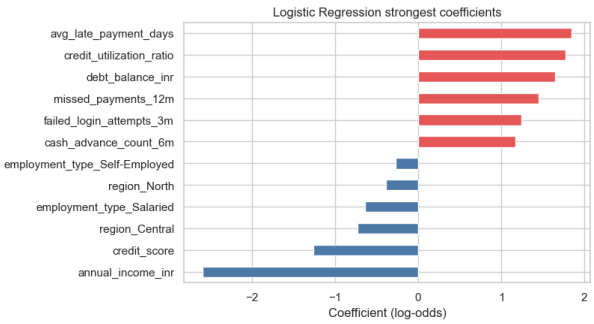


Figure 3: Strongest Logistic Regression coefficients

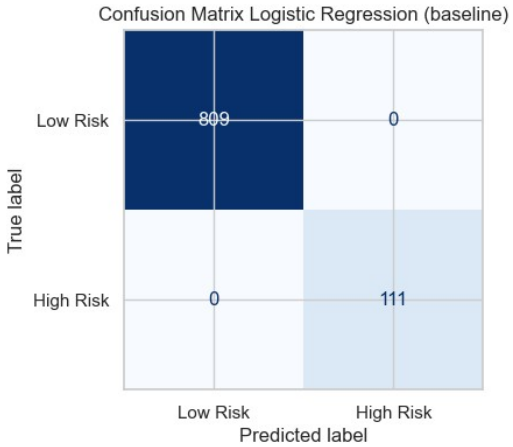


Figure 4: Confusion matrix of the baseline model

4.2 Confusion matrix, accuracy, precision, recall and F1-Score (Q11)

Metric	Training set	Test set
Accuracy	0.9970	1.0000
Precision (High Risk)	–	1.0000
Recall (High Risk)	–	1.0000
F1-Score (High Risk)	–	1.0000
AUC-ROC	–	1.0000

All 809 low-risk and all 111 high-risk customers in the test set were classified correctly. The training accuracy (0.997) is almost the same, so there is no overfitting. My honest conclusion is that this dataset is very clean and almost linearly separable, so even a simple Logistic Regression is already very strong. On real bank data I would expect lower numbers.

4.3 Type-I and Type-II errors from the confusion matrix (Q12)

TN	FP (Type-I)	FN (Type-II)	TP	FPR	FNR
809	0	0	111	0.000	0.000

The baseline made no Type-I errors (no safe customer was flagged) and no Type-II errors (no risky customer was missed). In general the Type-II errors are the costly ones for the bank, which is why Recall is my key metric in the next parts.

5. Part D: Handling Imbalanced Data

5.1 Impact of class imbalance on model performance (Q13)

To show the problem, I first built a dummy model that always predicts "Low Risk". It reaches 87.9% accuracy but its recall is 0, it misses all 111 risky customers and its AUC is only 0.5. This shows that accuracy alone is useless on imbalanced data.

Model	Test accuracy	Recall (High Risk)	AUC	FN
Dummy (always Low Risk)	0.8793	0.0000	0.5000	111
Logistic Regression (baseline)	1.0000	1.0000	1.0000	0

My Logistic Regression is already very strong on this clean data, so to show the real effect of imbalance I ran a stress test. I kept all low-risk training rows but only a fraction of the high-risk rows (average of 10 random draws) and measured Recall on the same test set. When the high-risk share of the training data fell from 12% to 0.1%, the recall dropped from 1.00 to 0.50.

High-risk rows kept	High-risk share of train	Recall (High Risk)	F1 (High Risk)
100%	0.1212	1.000	1.000
50%	0.0645	0.998	0.999
20%	0.0268	0.989	0.995
10%	0.0134	0.968	0.983
5%	0.0068	0.923	0.960
2%	0.0025	0.742	0.851
1%	0.0012	0.501	0.663

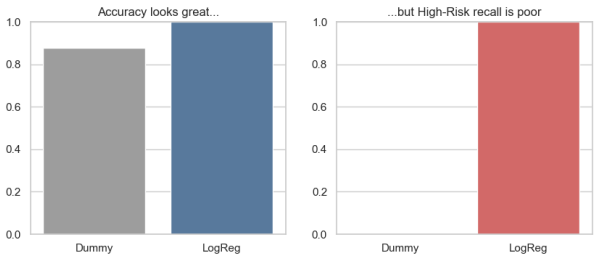


Figure 5: Dummy model vs Logistic Regression

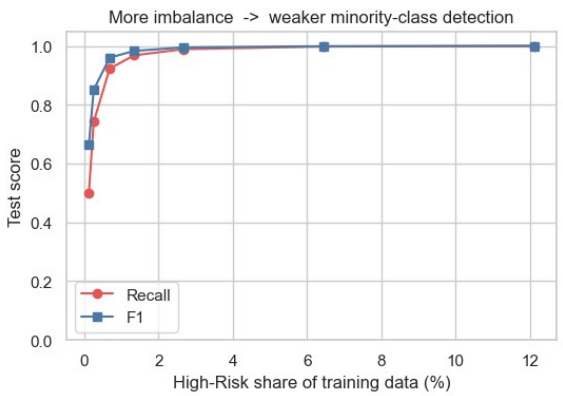


Figure 6: Stress test: recall vs imbalance

5.2 Under-Sampling, Over-Sampling, SMOTE and ADASYN (Q14)

I applied four techniques to the training data only, so the test set stayed untouched and the evaluation is honest. Random Under-Sampling removes majority-class rows (information is lost). Random Over-Sampling duplicates minority-class rows. SMOTE creates new synthetic minority rows by interpolating between neighbouring minority points. ADASYN is like SMOTE but creates more synthetic rows in the regions that are harder to learn. After each technique I retrained the Logistic Regression.

Technique	Low Risk rows	High Risk rows
Original training data	3,234	446

Technique	Low Risk rows	High Risk rows
Under-Sampling	446	446
Over-Sampling	3,234	3,234
SMOTE	3,234	3,234
ADASYN	3,234	3,211

5.3 Performance before and after balancing (Q15)

Model	Precision	Recall	F1	AUC	FP	FN
LogReg (baseline)	1.0000	1.0000	1.0000	1.0000	0	0
LogReg + Under-Sampling	0.9823	1.0000	0.9911	1.0000	2	0
LogReg + Over-Sampling	0.9823	1.0000	0.9911	1.0000	2	0
LogReg + SMOTE	0.9823	1.0000	0.9911	1.0000	2	0
LogReg + ADASYN	0.9652	1.0000	0.9823	1.0000	4	0

The baseline recall was already 1.00, so balancing cannot improve it. All four techniques kept the recall at 1.00 and the AUC at about 1.00, but they pushed the model to flag more customers, so precision fell a little (2 false positives for Under-Sampling, Over-Sampling and SMOTE, and 4 for ADASYN). So resampling is very useful when the risky class is rare, as the stress test shows, but on this clean dataset with 12% risky customers it gives no recall gain.

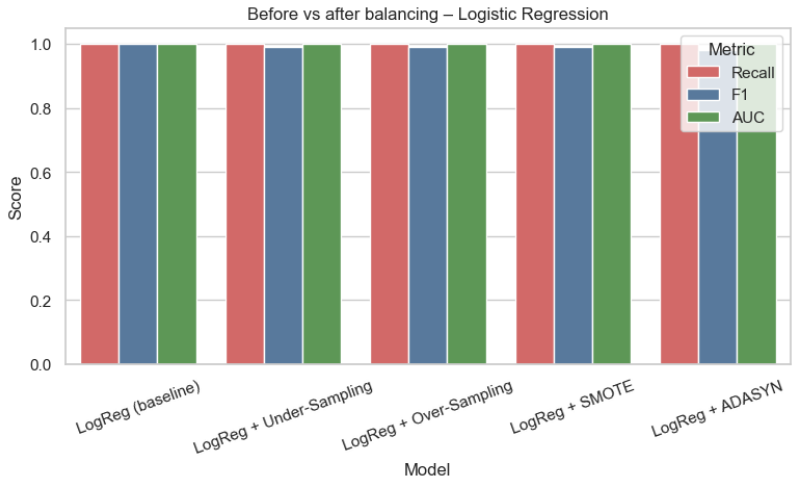


Figure 7: Recall, F1 and AUC before and after balancing

6. Part E: Tree-Based Classification Models

6.1 Decision Tree Classifier (Q16)

A Decision Tree asks a series of yes/no questions, for example "avg_late_payment_days > 12?", and ends in a leaf with a predicted class. I first trained an unrestricted tree. It grew to depth 17 with 74 leaves. It reached a test accuracy of 0.9707, but the recall for the High-Risk class was only 0.8378 (precision 0.9118, F1 0.8732). It missed 18 risky customers (FN) and wrongly flagged 9 safe ones (FP), so it is clearly weaker than Logistic Regression on this data.

6.2 Overfitting analysis (Q17)

The unrestricted tree has a training accuracy of 1.0000 against a test accuracy of 0.9707, a gap of 0.029. It memorised the training data, which is a clear case of overfitting. I also trained trees with max_depth from

1 to 25. The training score keeps going up towards 1.0, but the test F1 peaks near depth 8 (about 0.91) and then falls. So limiting the depth, or pruning, is needed.

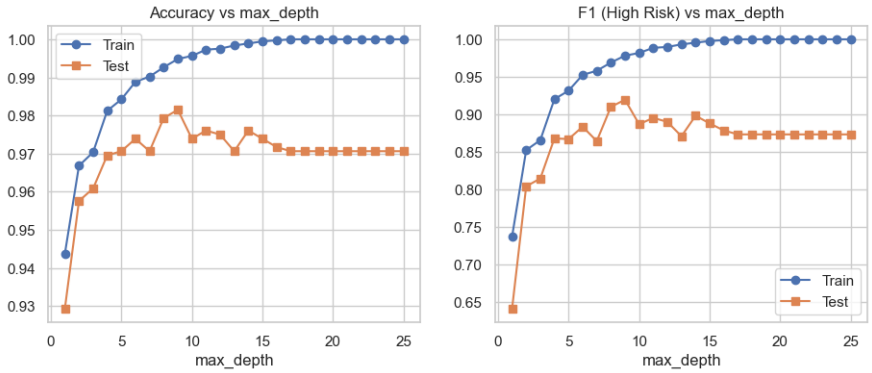


Figure 8: Train vs test accuracy and F1 for different tree depths

6.3 Random Forest Classifier (Q18)

A Random Forest builds many trees on random samples of rows and features and takes a majority vote. I used 200 trees. It reached a test accuracy of 0.9967, precision 1.0000, recall 0.9730, F1 0.9863 and AUC 1.0000, with only 3 false negatives and 0 false positives. The feature importance plot shows that avg_late_payment_days, missed_payments_12m, credit_utilization_ratio and credit_score are the top features, which matches what Logistic Regression found.

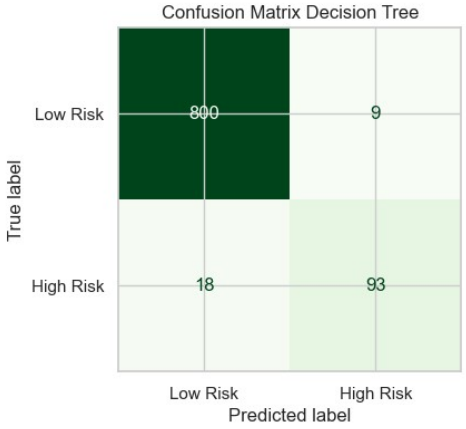


Figure 9: Confusion matrix of the Decision Tree

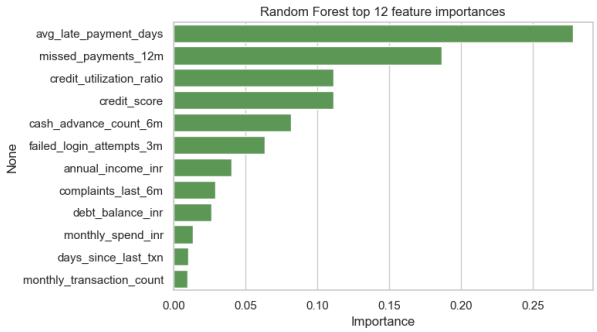


Figure 10: Random Forest feature importance

6.4 Decision Tree vs Random Forest (Q19)

Model	Train acc	Test acc	Precision	Recall	F1	AUC	Gap
Decision Tree	1.0000	0.9707	0.9118	0.8378	0.8732	0.9134	0.0293
Random Forest	1.0000	0.9967	1.0000	0.9730	0.9863	1.0000	0.0033

Random Forest is better on every metric, and the gap between training and test accuracy drops from 0.029 to 0.003. It averages many different trees, so the variance goes down and it generalises much better than a single tree. The single tree is easier to explain, but I would not use it alone.

7. Part F: Hyperparameter Tuning

7.1 Randomized Search CV (Q20)

I used 5-fold Stratified Cross-Validation and the F1-score of the High-Risk class as the scoring metric, because it balances precision and recall for the minority class. Randomized Search tried 40 random combinations for the Decision Tree and 25 for the Random Forest.

Model	Best parameters	CV F1
Decision Tree	max_depth = 6, min_samples_leaf = 5, min_samples_split = 5, criterion = entropy	0.9073
Random Forest	n_estimators = 500, max_depth = 15, min_samples_leaf = 8, min_samples_split = 2, max_features = log2, class_weight = balanced_subsample	0.9674

Tuning reduced the depth of the single tree, which reduces overfitting. The forest chose class weighting (balanced_subsample), which gives the minority class more weight in every tree.

7.2 Grid Search CV for the best model (Q21)

The Random Forest had the better CV score, so I fine-tuned it with Grid Search on a small grid around the best values: n_estimators 300 or 500, max_depth 10, 15 or None, min_samples_split 2 or 5, and min_samples_leaf 4 or 8. Grid Search selected the same parameters as Randomized Search and the CV F1 stayed at 0.9674. This is a good sign: it confirms that the randomized search had already found the best region, so there was no further gain.

7.3 Tuned vs untuned performance (Q22)

Model	Train acc	Test acc	Precision	Recall	F1	FP	FN
Decision Tree	1.0000	0.9707	0.9118	0.8378	0.8732	9	18
Decision Tree (tuned)	0.9894	0.9815	0.9123	0.9369	0.9244	10	7
Random Forest	1.0000	0.9967	1.0000	0.9730	0.9863	0	3
Random Forest (Randomized)	0.9959	0.9967	0.9737	1.0000	0.9867	3	0
Random Forest (Grid)	0.9959	0.9967	0.9737	1.0000	0.9867	3	0

Tuning helped most for the weak model: the Decision Tree recall went from 0.838 to 0.937 and the false negatives from 18 to 7. For the Random Forest the recall improved from 0.973 to 1.000 (3 false negatives became 0), at the cost of 3 false positives. For a bank this is a good trade. Grid Search gave exactly the same test result as Randomized Search.

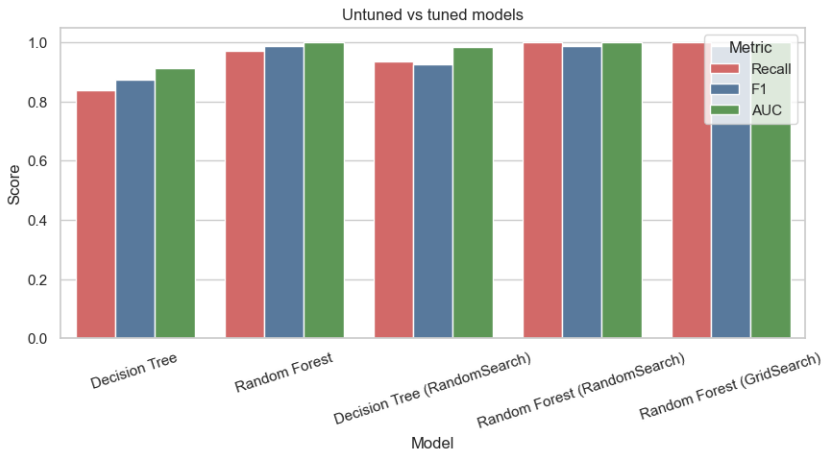


Figure 11: Recall, F1 and AUC of untuned and tuned models

8. Part G: Model Evaluation and ROC Analysis

8.1 ROC curves of all models (Q23)

I plotted the ROC curve of every model. All Logistic Regression variants and the Random Forests stay at the top-left corner (TPR close to 1 at FPR close to 0), which means almost perfect separation. The untuned Decision Tree is clearly lower, and the tuned one is slightly lower. The dummy model would lie on the diagonal line.

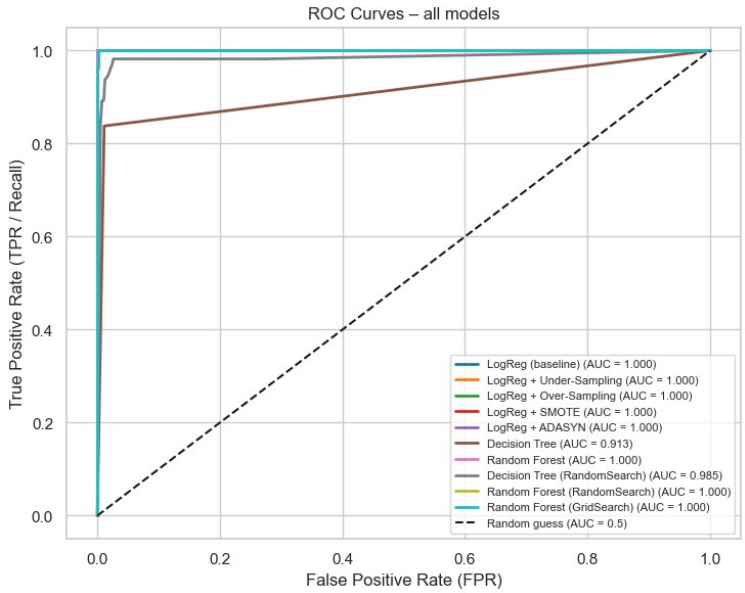


Figure 12: ROC curves of all models

8.2 AUC-ROC scores (Q24)

Model	AUC	Recall	Precision	F1
LogReg (baseline)	1.0000	1.0000	1.0000	1.0000
LogReg + Under / Over-Sampling / SMOTE	1.0000	1.0000	0.9823	0.9911
LogReg + ADASYN	1.0000	1.0000	0.9652	0.9823
Random Forest	1.0000	0.9730	1.0000	0.9863
Random Forest (tuned)	1.0000	1.0000	0.9737	0.9867
Decision Tree (tuned)	0.9847	0.9369	0.9123	0.9244
Decision Tree	0.9134	0.8378	0.9118	0.8732
Dummy (always Low Risk)	0.5000	0.0000	0.0000	0.0000

Many models tie at an AUC of about 1.00 on this single test split, so AUC alone cannot choose the winner. I therefore also looked at recall, precision, the number of errors and cross-validation.

8.3 Selecting the best final model (Q25)

My business requirement is to minimise false negatives. So instead of accuracy I ranked the models with the F2-score, which counts recall twice as much as precision, and with an assumed cost where one false negative costs 10 units and one false positive costs 1 unit.

Model	FP	FN	Business cost
LogReg (baseline)	0	0	0
LogReg + Under / Over-Sampling / SMOTE	2	0	2

Model	FP	FN	Business cost
Random Forest (Randomized / Grid)	3	0	3
LogReg + ADASYN	4	0	4
Random Forest	0	3	30
Decision Tree (tuned)	10	7	80
Decision Tree	9	18	189

Because the test set has only 111 risky customers, one or two customers can change the result. So I compared the best candidates again with repeated 5-fold cross-validation (2 repeats) on the training data.

Model	Recall	Precision	F1	F2	AUC
LogReg (baseline)	0.9799	0.9899	0.9848	0.9818	0.9995
LogReg + Under-Sampling	0.9855	0.9446	0.9644	0.9769	0.9994
LogReg + SMOTE	0.9821	0.9694	0.9755	0.9794	0.9994
Random Forest (Grid Search)	0.9653	0.9697	0.9674	0.9661	0.9979

The Logistic Regression baseline has the best cross-validated F2 (0.982). It is as good as the complex models, the simplest to explain and the fastest. Finally I tuned the decision threshold on out-of-fold predictions of the training data, so the test set stayed untouched. The best F2 was at a threshold of 0.31 instead of the default 0.5. A lower threshold gives extra safety for recall, and on the test set the model still has Recall = 1.00, Precision = 1.00 and AUC = 1.00.

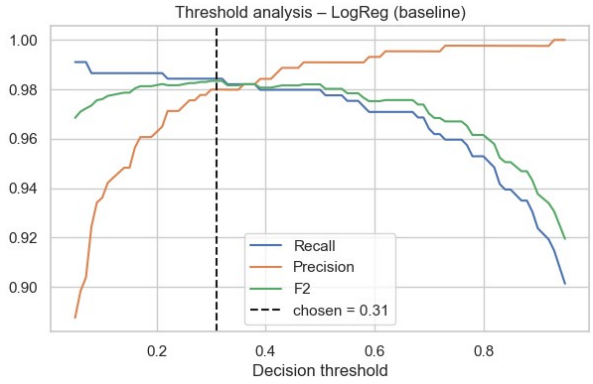


Figure 13: Threshold analysis of the final model

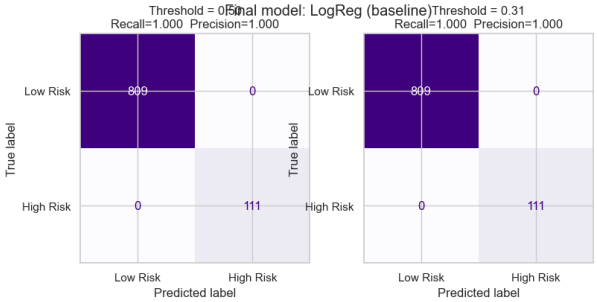


Figure 14: Final model: threshold 0.50 vs 0.31

9. Part H: Final Analysis

Best model and justification (Q26)

The best model is Logistic Regression (no resampling) with KNN-imputed and scaled features and a decision threshold of 0.31. On the test set it gives Accuracy, Precision, Recall, F1 and AUC of 1.00, with 0 false positives and 0 false negatives, and it has the best cross-validated F2 of 0.982. It is also simple, fast and explainable, so there is no need for a heavier model.

Impact of imbalance handling

Only 12.1% of the customers are high risk. The dummy model gets 87.9% accuracy with 0% recall, and in the stress test the recall fell from 1.00 to 0.50 when the risky share dropped to 0.1%. On the real dataset the baseline was already perfect, so the four balancing techniques kept the recall at 1.00 but added 2 to 4

false positives (F1 between 0.98 and 0.99). So resampling is a safety net for rarer risk events, not a must-have here.

Comparison of performance metrics

Model	Recall	Precision	F1	AUC	FN	FP
Logistic Regression (baseline)	1.000	1.000	1.000	1.000	0	0
LogReg + Under / Over-Sampling / SMOTE	1.000	0.982	0.991	1.000	0	2
LogReg + ADASYN	1.000	0.965	0.982	1.000	0	4
Decision Tree	0.838	0.912	0.873	0.913	18	9
Decision Tree (tuned)	0.937	0.912	0.924	0.985	7	10
Random Forest	0.973	1.000	0.986	1.000	3	0
Random Forest (Randomized / Grid Search)	1.000	0.974	0.987	1.000	0	3

Business interpretation of false positives and false negatives

A false negative means a high-risk customer is approved or not monitored, so the bank gets a default or fraud loss with no early warning. This is the expensive mistake, so recall is the priority. A false positive means a safe customer is flagged, which causes an unnecessary manual review and maybe some customer annoyance, a small cost. With an assumed cost ratio of 10 : 1, the baseline Logistic Regression has the lowest total cost (0) and the untuned Decision Tree the highest (189).

Recommendations

- Deploy Logistic Regression with a threshold of about 0.31 as the early-warning model and review the flagged customers manually.
- Monitor recall and the high-risk share every month. If risky customers become rarer, retrain with SMOTE or class weights.
- The main risk drivers to watch are late-payment days, missed payments, credit utilisation, debt balance and low credit score or income.
- This dataset is very clean and almost linearly separable, so on real production data I expect lower scores and would re-validate with a time-based split.

10. Streamlit App and Conclusion

Streamlit app and submission (Q27)

I saved the final pipeline (KNN imputation, scaling, one-hot encoding and Logistic Regression with threshold 0.31) as a joblib file and built a Streamlit app (app.py). It has three tabs: Single customer (enter the details and get a risk probability), Batch prediction (upload a CSV and download the predictions) and Model results (all comparison tables and the ROC curve). The screenshots below show a typical customer predicted as low risk and a customer with many late payments and a low credit score predicted as high risk.

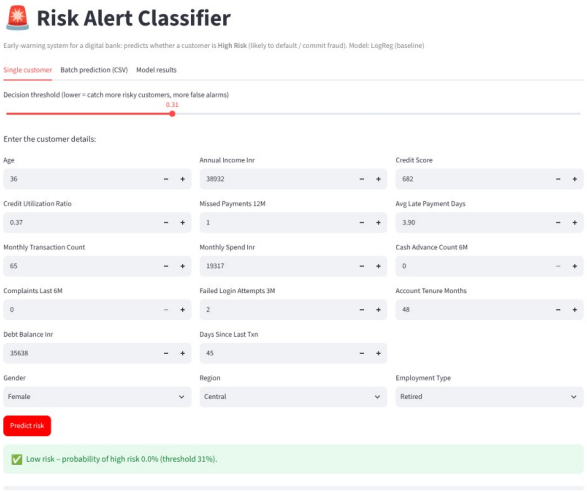


Figure 15: App: customer predicted Low Risk

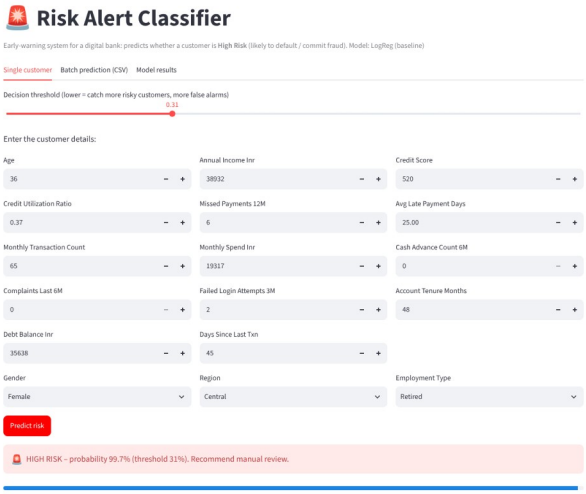
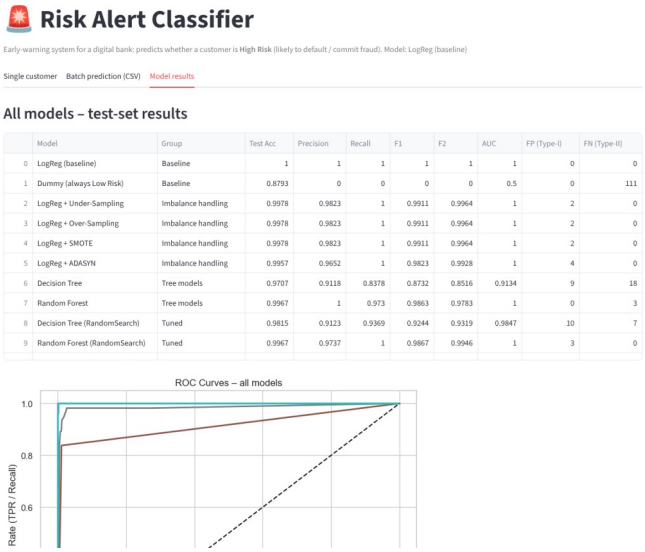


Figure 16: App: customer predicted High Risk



	Model	Group	Test Acc	Precision	Recall	F1	F2	AUC	FP (Type-I)	FN (Type-II)
0	LogReg (baseline)	Baseline	1	1	1	1	1	1	0	0
1	Dummy (always Low Risk)	Baseline	0.8793	0	0	0	0	0.5	0	111
2	LogReg + Under-Sampling	Imbalance handling	0.9978	0.9823	1	0.9911	0.9964	1	2	0
3	LogReg + Over-Sampling	Imbalance handling	0.9978	0.9823	1	0.9911	0.9964	1	2	0
4	LogReg + SMOTE	Imbalance handling	0.9978	0.9823	1	0.9911	0.9964	1	2	0
5	LogReg + ADASYN	Imbalance handling	0.9957	0.9652	1	0.9823	0.9928	1	4	0
6	Decision Tree	Tree models	0.9707	0.9118	0.8378	0.8732	0.8516	0.9134	9	18
7	Random Forest	Tree models	0.9967	1	0.973	0.9863	0.9783	1	0	3
8	Decision Tree (RandomSearch)	Tuned	0.9815	0.9123	0.9369	0.9244	0.9319	0.9847	10	7
9	Random Forest (RandomSearch)	Tuned	0.9967	0.9737	1	0.9867	0.9946	1	3	0

Figure 17: App: Model results tab